

TCH017 – REPRÉSENTATION INTERNE DES NOMBRES ENTIERS

par

IANNICK GAGNON

PROFESSEUR ENSEIGNANT

DÉPARTEMENT DES ENSEIGNEMENTS GÉNÉRAUX

MONTREAL, LE 12 SEPTEMBRE 2026

ÉCOLE DE TECHNOLOGIE SUPÉRIEURE
UNIVERSITÉ DU QUÉBEC



La qualité de ce document est importante pour nous. Veuillez
communiquer toute erreur à iannick.gagnon@etsmtl.ca.

TABLE DES MATIÈRES

1.	Motivation.....	3
2.	Les entiers non signés	4
3.	Les entiers signés.....	5
4.	La représentation signe-valeur	5
5.	La représentation biaisée.....	7
5.1.	Exercices.....	8
5.2.	Une remarque sur la compacité	8
6.	Le complément à un.....	9
6.1.	Exercices.....	11
7.	Le complément à deux.....	12
7.1.	Exercices.....	13
8.	Résumé	14

Représentation des nombres entiers

Dans le document intitulé « TCH017 – Systèmes de numération », nous avons abordé différents ensembles de symboles et d'opérations regroupés sous le nom de **systèmes de numération**. En particulier, nous avons étudié le système **binaire** à base 2, le système **octal** à base 8, le système **décimal** à base 10 et le système **hexadécimal** à base 16.

Ce document s'inspire du fait que les ordinateurs, en tant que machines électroniques, sont intrinsèquement binaires. Autrement dit, ils le comprennent que les zéros et les uns sous la forme de courants électriques qui passent (1) ou ne passent pas (0).

1. Motivation

Les ordinateurs traitent des données variées, incluant des nombres, des instructions, des images, des sons, et plus encore. Cependant, à leur niveau fondamental, toutes ces informations sont exprimées en forme binaire. L'unité d'information élémentaire est donc le chiffre binaire (0 ou 1) appelé **bit**. Une information plus complexe, comme un chiffre ou un caractère correspond à un **ensemble de bits**.

Le fait de bien comprendre la représentation interne des informations (p. ex. les nombres) permet d'écrire du code plus efficace, plus sécuritaire et plus facile à transporter d'une plateforme à l'autre.

Cela pourrait aussi vous éviter l'angoisse qui est parfois causée par le phénomène suivant :

```
#include <stdio.h>
#include <limits.h>

int main()
{
    int x = INT_MAX;

    printf("a = %i\n", x);
    printf("a + 1 = %i\n", x + 1);

    getchar();
    return 0;
}
```

```
a = 2147483647
a + 1 = -2147483648
```

Continuez de lire!

2. Les entiers non signés

Les entiers non signés sont ceux qui sont **strictement positifs ou nuls** qui varient de **0 à $2^n - 1$** avec n qui représente le nombre de bits disponibles pour le codage. Ainsi, un entier non signé codé sur 8 bits ou 1 **octet** peut représenter les valeurs allant de 0 à $2^8 - 1 = 255$. La valeur 0 correspond à 0b00000000 et la valeur 255 correspond à 0b11111111. Le préfixe « 0b » est une convention utilisée pour signaler qu'il s'agit d'un nombre binaire. Nous nous rappelons qu'il est possible de convertir en base 10 en prenant la somme pondérée des puissances successives de la base, soit 2 pour le binaire, **de la droite vers la gauche** en débutant à zéro et en s'arrêtant à $n - 1$. Pour un entier codé sur 8 bits, cela donne $\sum_{i=0}^7 a_i 2^i$ avec a_i qui représente le i -ème chiffre binaire toujours à partir de la droite. Le binaire 0b11111111 vaut donc effectivement 255 comme le montre le calcul suivant :

i	a_i	2^i	$a_i 2^i$
0	1	$2^0 = 1$	1
1	1	$2^1 = 2$	2
2	1	$2^2 = 4$	4
3	1	$2^3 = 8$	8
4	1	$2^4 = 16$	16
5	1	$2^5 = 32$	32
6	1	$2^6 = 64$	64
7	1	$2^7 = 128$	128
Σ			255

Quelle est la plage de valeurs que peut représenter un entier non signé codé sur 2 octets?

Quel est le plus petit nombre de bits nécessaires pour encoder la valeur $10! = 1 \cdot 2 \cdot \dots \cdot 9 \cdot 10$ dans un entier non signé? Sur la TI-Nspire, la factorielle (!) correspond à : menu > Probability (5) > Factorial (1).

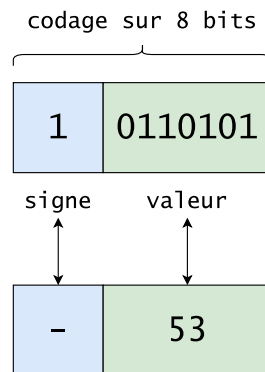
3. Les entiers signés

Les entiers non signés ont l'avantage d'être compacts pour les entiers positifs (ceci sera expliqué un peu plus loin), mais ils ont le « désavantage » de ne pas pouvoir représenter les valeurs négatives. Dans cette section, nous dévoilons les quatre représentations suivantes qui supportent les valeurs négatives :

1. Signe-valeur
2. Biaisée
3. Complément à un
4. Complément à deux

4. La représentation signe-valeur

Le moyen le plus intuitif d'inclure la représentation du signe est d'utiliser un des bits pour l'encoder. Par convention, ce sera toujours le bit le plus à gauche de la représentation binaire. **Lorsque le bit le plus à gauche est égal à un, la valeur est négative, sinon elle est positive.** La plus petite valeur représentée par un entier en format signe-valeur codé sur 8 bits est donc 0b11111111 (-127) et 0b01111111 (127). Voici une représentation schématique de la valeur 0b10110101 et de sa conversion en valeur décimale :



Remarquez que les flèches vont dans les deux sens pour schématiser l'encodage et le décodage. **Le format signe-valeur commence par le signe à gauche suivi de la valeur par la méthode des divisions répétées.**

Quelle est la plage de valeurs que peut représenter un entier en format signe-valeur codé sur 5 bits?

De manière générale, la plage des valeurs représentables correspond à $[-(2^{n-1} - 1), 2^{n-1} - 1]$.

Encodez les valeurs suivantes en format signe-valeur sur 10 bits :

-23_{16}	89_{10}	-71_8

Bien qu'intuitive, la représentation signe-valeur a la particularité d'avoir **deux représentations distinctes du zéro**, soit 0b10000000 et 0b00000000 sur 8 bits par exemple. **Il y a donc un zéro négatif et un zéro positif**. Cela pourrait facilement porter à confusion et c'est aussi moins efficace. Le désavantage le plus sérieux de cette représentation est qu'elle rend les calculs inutilement complexes. Par exemple, l'addition de -7 (0b10000111) et 3 (0b00000011) donne :

$$\begin{array}{r} 1\ 0\ 0\ 0\ 1\ 1\ 1 \\ 0\ 0\ 0\ 0\ 0\ 1\ 1 \\ \hline 1\ 0\ 0\ 1\ 0\ 1\ 0 \end{array}$$

Autrement dit $-7 + 3$ donne -10 et non pas -4 . Il faudrait donc que le matériel (p. ex. le processeur) puisse distinguer le zéro positif du zéro négatif en plus de tenir compte des combinaisons de signes des opérandes lors des calculs : (+, +), (+, -), (-, +) et (-, -).

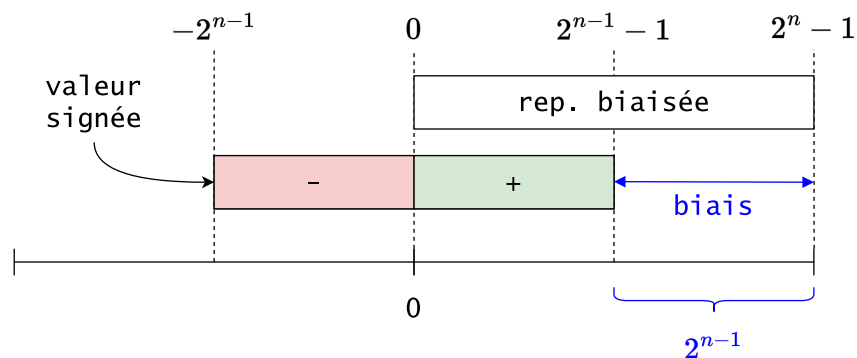
5. La représentation biaisée

L'idée derrière la représentation biaisée est d'utiliser la moitié de la plage de valeurs pour représenter les valeurs négatives et l'autre pour les valeurs positives. Pour un entier non signé codé sur n bits, le nombre de valeurs représentables est de 2^n sur la plage qui va de 0 à $2^n - 1$. Le -1 découle de l'inclusion du zéro. Par exemple, avec trois bits, nous avons 0b000 (0), 0b001 (1), 0b010 (2), 0b011 (3), 0b100 (4), 0b101 (5), 0b110 (6) et 0b111 (7), soit $2^3 = 8$ valeurs distinctes.

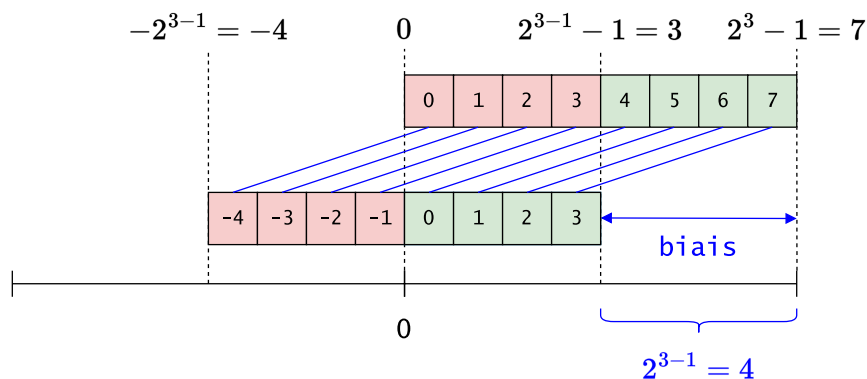
Ainsi, nous divisons la plage en deux faisant que $2^n/2 = 2^{n-1}$ valeurs négatives se trouvent à gauche du zéro et que $2^{n-1} - 1$ valeurs positives se trouvent à droite. Le -1 découle de l'inclusion du zéro dans la partie positive de droite par convention. Par exemple, avec trois bits :

Valeur négatives				Valeurs positives			
0b000	0b001	0b010	0b011	0b100	0b101	0b110	0b111
-4	-3	-2	-1	0	1	2	3

Voici une représentation schématisque du passage de la représentation signée bits à la représentation biaisée sur n bits :



La plage de valeurs que peut représenter un entier en format biaisé est donc $[-2^{n-1}, 2^{n-1} - 1]$. Pour $n = 3$, cela donne de $-2^{3-1} = -4$ à $2^{3-1} - 1 = 3$, donc $[-4, 3]$ ou $[-4, -3, -2, -1, 0, 1, 2, 3]$ pour reprendre le jeu de couleurs du tableau précédent. Voici le schéma correspondant :



Pour encoder la valeur -98 en représentation biaisée sur $n = 8$ bits, nous commençons par calculer le biais $2^{8-1} = 128$ et nous l'additionnons : $-98 + 128 = 30$. Nous faisons ensuite la conversion en base 2 par divisions répétées sur 8 bits : 0b00011110.

Pour passer de la représentation biaisée à la valeur décimale, nous convertissons la valeur biaisée en base 10 et **soustrayons** le biais :

$$(2^4 + 2^3 + 2^2 + 2^1) - 128 = (16 + 8 + 4 + 2) - 128 = -98$$

Voici la procédure pour la conversion sur n bits avec un biais de 2^{n-1} :

Étape	entier $\rightarrow$ biaisée	biaisée $\rightarrow$ entier
1	Calculer le biais	Calculer le biais
2	Additionner le biais	Convertir en base 10
3	Convertir en base 2	Soustraire le biais

5.1. Exercices

a) Convertissez les entiers suivants en représentation biaisée :

- 117 sur 8 bits
- 12598 sur 16 bits
- -79 sur 8 bits
- -11015 sur 16 bits

b) Convertissez les nombres en représentation biaisée suivants en entiers non signés :

- 15 sur 8 bits
- 216 sur 8 bits
- 576 sur 16 bits
- 53512 sur 16 bits

5.2. Une remarque sur la compacité

Nous avons mentionné plus tôt que les entiers non signés sont « compacts » pour la représentation d'entiers positifs. Nous pouvons maintenant clarifier ce commentaire. Si nous disposons de n bits, la plus grande valeur qu'un entier non signé peut représenter est $2^n - 1 = 2^8 - 1 = 255$ alors que celle d'un entier signé en représentation biaisée est $2^{n-1} - 1 = 2^{8-1} - 1 = 126$.

Cela peut sembler sans conséquences, mais si un-e programmeur-e choisit un entier signé pour représenter des valeurs qui sont bornées entre 0 et 255 comme les valeurs de pixels RGB par exemple, il ou elle devra utiliser un minimum

de $2^{n-1} - 1 = 255 \Rightarrow n = 9$ bits. Il est fort probable que le langage de programmation utilisé ne possède que des entiers de 8, 16, 32 et/ou 64 bits. En fin de compte, il ou elle utilisera les entiers de 16 bits plutôt que 8 et gaspillera au moins 50% de la mémoire utilisée.

6. Le complément à un

La représentation biaisée fonctionne, mais elle contient des étapes intermédiaires coûteuses. Cette section présente la méthode du complément à un qui représente un nombre négatif par **l'inverse binaire de sa valeur absolue** avec le bit de gauche comme **bit de signe** (0 = positif, 1 = négatif). Autrement dit, pour trouver l'inverse de 5_{10} sur 4 bits, nous prenons le binaire 0b0101 et nous inversons chacun des bits : 0b1010. La valeur -5_{10} correspond donc à 0b1010 en complément à un. À quelle valeur décimale correspond la valeur en complément à un 0b1011? Puisque le bit de gauche signale qu'il s'agit d'un nombre négatif, nous inversons les bits : 0b0100. Cette valeur correspond à 4_{10} . Il s'agit donc de -4_{10} . À quelle valeur décimale correspond la valeur en complément à un 0b0111? Puisqu'il s'agit d'une valeur positive, nous faisons la conversion directement : $2^2 + 2^1 + 2^0 = 7_{10}$.

En résumé :

1. Si la valeur est positive / que le bit de signe est égal à zéro, faire la conversion directement.
2. Si la valeur est négative / que le bit de signe est égal à un, inverser le binaire et faire la conversion.

Voici une représentation schématique sur $n = 3$ bits avec les inverses binaires reliés par des flèches qui montre aussi d'où vient le zéro négatif :

+0	0	0	0	
+1	0	0	1	
+2	0	1	0	
+3	0	1	1	
-3	1	0	0	←
-2	1	0	1	←
-1	1	1	0	←
-0	1	1	1	←

De manière générale, la plage des valeurs représentables correspond à $[-(2^{n-1} - 1), 2^{n-1} - 1]$ comme pour la représentation signe-valeur.

Voici quelques exemples sur 4 bits :

-6_{10}	7_{10}	0b1111	0b0000

Le complément à un a l'avantage que les opérations mathématiques se font directement en binaire. Considérez le premier exemple avec 6_{10} (0b0101) et -6_{10} (0b1010) :

$$\begin{array}{r}
 0b0101 + \\
 0b1010 \\
 \hline
 0b1111 \quad (-0)
 \end{array}$$

Nous constatons que la somme de deux opposés donne effectivement zéro. Prenons maintenant -4_{10} (0b1011) et -1_{10} (0b1110) avec les retenues en **bleu** :

$$\begin{array}{r}
 \text{11} \\
 0b1011 + \\
 0b1110 \\
 \hline
 0b?001
 \end{array}$$

Il semble y avoir une impasse, car les deux bits de gauches sont déjà activés et il y a aussi une retenue. **La règle prévoit que la retenue sera ajoutée au bit de droite** (en **vert**) :

$$\begin{array}{r}
 \text{1111} \\
 0b1011 + \\
 0b1110 \\
 \hline
 0b1010 \quad (-5)
 \end{array}$$

La valeur obtenue 0b1010 est négative et son inverse donne 0b0101 qui vaut 5_{10} , ce qui signifie qu'il s'agit de -5_{10} . Autrement dit : $-4_{10} + (-1_{10}) = -5_{10}$ tel qu'attendu.

En résumé, le complément à un possède l'avantage qu'il facilite les opérations mathématiques, mais il engendre une perte d'expressivité par l'usage de deux symboles pour le zéro et une mécanique de retenue coûteuse.



En complément à un, la retenue d'une somme sur le bit le plus à gauche est transféré au bit le plus à droite.

6.1. Exercices

a) Convertissez les valeurs suivantes en complément à un :

- 105_{10} sur 8 bits
- 12768_{10} sur 16 bits
- -105_{10} sur 16 bits
- -22098 sur 16 bits

b) Convertissez les valeurs en complément à un suivantes en valeurs décimales :

- 1101 1010
- 1010 1101 0110 0000
- 0010 1000
- 0110 1101 1001 0010

7. Le complément à deux

La méthode du complément à deux consiste à représenter un nombre négatif par l'inverse binaire de sa valeur absolue (c.-à-d. le complément à un) **plus un**. Lorsque nous utilisons ce format et que nous **ignorons la retenue** (s'il y en a) **du bit le plus à gauche** découlant d'une somme, le zéro double est éliminé comme le montre le schéma suivant :

0	0	0	0
+1	0	0	1
+2	0	1	0
+3	0	1	1
-4	1	0	0
-3	1	0	1
-2	1	1	0
-1	1	1	1

Par exemple, pour convertir -2_{10} en complément à deux sur 3 bits, nous commençons par prendre la valeur absolue et nous la transformons en binaire, soit 0b010 qui donne 0b101 lorsqu'il est inversé. Finalement nous additionnons une unité : $0b101 + 0b001 = 0b110$. La valeur -2_{10} correspond donc à 0b110 en complément à deux. En calculant la somme de 0b010 (2_{10}) et 0b110 (-2_{10}) **en ignorant la retenue de gauche**, nous trouvons zéro tel qu'attendu.

Pour faire le chemin inverse, on prend toujours le complément à un suivi de l'addition d'une unité : $0b110 \rightarrow 0b001 + 0b001 = 0b010$. Cette valeur correspond à $2^1 = 2_{10}$ tel qu'attendu.

Reprenons l'exemple problématique sur le complément à un, soit $-4_{10} + (-1_{10})$ sur 4 bits :

Obtenir le binaire des valeurs absolues :

$$|-4_{10}| = 4_{10} \rightarrow 0b0100$$

$$|-1_{10}| = 1_{10} \rightarrow 0b0001$$

Prendre le complément à un :

$$0b0100 \rightarrow 0b1011$$

$$0b0001 \rightarrow 0b1110$$

Additionner une unité :

$$0b1011 + 0b0001 = 0b1100$$

$$0b1110 + 0b0001 = 0b1111$$

Calculer la somme en ignorant la retenue à gauche :

$$\begin{array}{r} 1 \\ 0b1100 + \\ 0b1111 \\ \hline 0b1011 \text{ (-5)} \end{array}$$

Il s'agit d'une valeur négative puisque le bit à gauche est activé. La conversion en valeur décimale commence par l'inversion suivie de l'addition sans oublier de faire la conversion en valeur négative : $0b1011 \rightarrow 0b0100 + 0b0001 = 0b0101 = -5_{10}$.



En complément à deux, la retenue d'une somme sur le bit le plus à gauche est ignorée.

En résumé pour convertir une valeur en complément à deux dans les deux sens :

1. Si la valeur est positive, faire la conversion directement.
2. Si la valeur est négative, prendre le complément à un et additionner une unité.

7.1. Exercices

a) Convertissez les valeurs suivantes en complément à deux :

- 105_{10} sur 8 bits
- 12768_{10} sur 16 bits
- -105_{10} sur 16 bits
- -22098_{10} sur 16 bits

b) Convertissez les valeurs en complément à deux suivantes en valeurs décimales :

- $1101\ 1010_2$
- $1010\ 1101\ 0110\ 0000_2$
- $0010\ 1000_2$
- $0110\ 1101\ 1001\ 0010_2$

8. Résumé

Cette section résume les éléments importants à retenir pour les évaluations sommatives.

Plages de valeurs pour un encodage sur n bits

- Un entier **non signé** peut encoder les valeurs suivantes : $[0, 2^n - 1]$.
- Un entier **signe-valeur** peut encoder les valeurs suivantes : $[-(2^{n-1} - 1), 2^{n-1} - 1]$.
- Un entier en **représentation biaisée** peut encoder les valeurs suivantes : $[-2^{n-1}, 2^{n-1} - 1]$.
- Un entier en **complément à un** peut encoder les valeurs suivantes : $[-(2^{n-1} - 1), 2^{n-1} - 1]$.
- Un entier en **complément à deux** peut encoder les valeurs suivantes : $[-2^{n-1}, 2^{n-1} - 1]$.

Précision sur la représentation signe-valeur

- Le bit le plus à gauche correspond au signe selon la convention suivante : $1 \rightarrow -, 0 \rightarrow +$.

Précisions sur la représentation biaisée

- Le biais vaut 2^{n-1}
- La conversion se fait selon les étapes suivantes :

Étape	entier $\rightarrow$ biaisée	biaisée $\rightarrow$ entier
1	Calculer le biais	Calculer le biais
2	Additionner le biais	Convertir en base 10
3	Convertir en base 2	Soustraire le biais

Précisions sur le complément à un

- Le bit le plus à gauche correspond au signe selon la convention suivante : $1 \rightarrow -, 0 \rightarrow +$.
- La conversion se fait directement pour les valeurs positives.
- Pour les valeurs négatives, on prend l'inverse binaire de la valeur absolue.

Précisions sur le complément à deux

- Le bit le plus à gauche correspond au signe selon la convention suivante : $1 \rightarrow -, 0 \rightarrow +$.
- La conversion se fait directement pour les valeurs positives.
- Pour les valeurs négatives, nous prenons le complément à un et additionnons une unité.